

Senior Project:
Makerspace Information Display
Final Report

Anna Noto, Matthew Bernardino, Jake Tandy

ECPE 196 - Spring 2026

Abstract.....	1
Project Objectives.....	2
Hardware Requirements.....	2
Software Requirements.....	3
Power & Cooling Requirements.....	3
Constraints.....	3
Problem Overview.....	4
Background.....	4
Design Implementation.....	5
Team Roles.....	5
System Structure.....	5
Hardware Description.....	5
Software Description.....	6
Timeline.....	7
Verification Procedure.....	8
Evaluation.....	10
Analysis.....	11
Budget.....	12
Labor Estimates.....	13
Global, Social, & Environmental Impact.....	13
Social Impact.....	13
Economic Impact.....	14
Environmental Impact.....	14
Future Work.....	15
Performance & Speed.....	15
Display Scale & Content.....	15
Additional Software Integrations.....	15
Conclusions.....	15
Acknowledgements.....	16
References.....	16
Appendix.....	17
User Manual.....	17
Poster.....	17
Charts, Diagrams, & Code.....	17

Abstract

This project presents the design and implementation of a centralized digital display system for the Makerspace to improve communication among employees, students, and faculty. Existing methods of sharing information are often fragmented and inconsistent, making it difficult for users to access real-time updates on space status, policies, resources, and announcements. To address this issue, a modular LED matrix display system was developed using a Raspberry Pi and a series of daisy-chained panels. The system integrates with Google Sheets as a dynamic data source, allowing content to be updated remotely and reflected on the display in near real-time. In response to stakeholder feedback, a Kivy-based touchscreen controller on a Raspberry Pi 3B improves accessibility for updating tool counts, machine shop status (open/closed), and hours, syncing with Google Sheets to trigger the display pipeline. The controller serves as an optional add-on, with full functionality retained through direct Google Sheets updates.

The software pipeline retrieves and processes data using Python-based tools, converting text into pixel-based images that are rendered across the LED matrix. The hardware system includes a structured power distribution network with integrated safety features and cooling mechanisms to support continuous operation. Initial testing has demonstrated successful panel communication, stable power delivery, and effective data rendering from external sources. This project highlights a scalable and adaptable solution for improving real-time communication in shared work environments, with potential applications in other collaborative spaces.

Project Objectives

Design and implement a centralized digital display system that streamlines communication between Makerspace employees, students, and faculty by providing clear, real-time information on space status, policies, resources, and announcements.

Functional Requirements

The functional requirements for this system focus on what is needed for the full system to operate properly from input to display. The system needs a stable 5V power supply to drive the LED panels, along with proper power delivery to the Raspberry Pi. WiFi connectivity is required so the system can access Google Sheets, which is the main source of display data. A working Google service account and API credentials must be set up for the software to read and write data using gspread. The Raspberry Pi must be configured with the right libraries and set to automatically run the program on boot so the display is always active. In addition, proper hardware connections such as the HUB75 header connector and a shared ground between the Pi and power supply are required to ensure reliable communication to correctly display information.

Hardware Requirements

The hardware subsystem must support a modular LED panel configuration that is scalable to accommodate future expansion. The power system is required to deliver stable voltage to all panels simultaneously, and must prevent overheating and overcurrent conditions through appropriate fusing and thermal management. Reliable GPIO connections and signal integrity between the Raspberry Pi and the panel array must be maintained throughout continuous operation, and the system must operate stably without crashes for periods exceeding 24 hours. The Raspberry Pi 4B is specified as the target compute platform for final deployment. From an environmental and physical standpoint, the system must operate safely within the Makerspace environment, incorporate both active and passive cooling to prevent overheating, implement safe cable management practices, and be housed in a durable mounting and enclosure structure.

Software Requirements

The software for this senior project is designed to control the LED matrix display by taking information from Google Sheets and converting that information into a format that can be displayed on the RGB matrix panels. The main software description describes a data pipeline where Google Sheets stores the display messages, gspread reads the selected cells, Pillow converts the text into an image, and the RGB matrix library outputs the image to the physical display. The software must be made into clear functions so each part of the program has its own purpose. One function handles the Google Sheets connection and retrieves the updated data, another function formats the text for the correct panel layout, another function creates the image using Pillow, and the final function sends the completed image to the LED matrix. Within the user manual we also explicitly state the files included and how they should run. We also explain steps to troubleshoot in case there is an issue with the device.

Power & Cooling Requirements

The power system for this senior project relies on a 5V DC power supply, which supplies the 18 panels with ample current. The power supply itself, the LRS-600, converts the 120V AC from the wall outlet to the 5V DC for distribution. The supply feeds into 3 separate rows of panels, each with identical circuitry. The power supply connects directly to a 30A fuse, which then distributes out to the panels using terminal blocks to keep connections secure and allow us to wire directly to the pre-existing panel wires. The ground for the circuits works in a similar way, starting at the power supply and using the same terminal blocks to run out to the panels themselves. Both the LED's and power supply generate heat during operation, which is negated by the power system's internal fan and regulated airflow. Providing the back area of the project with adequate airflow allows for the system to stay in thermal equilibrium. This is key for ensuring that the electronic components continue to function correctly.

Constraints

Power Constraints

- Limited to 5V 120 Amp PSU for LED panels
- Battery-powered controller limited to 6–7 hours runtime

Hardware Constraints

- Fixed panel size (64×64 per panel)
- HUB75 interface limits wiring and chaining direction
- Raspberry Pi GPIO and performance limitations

Software Constraints

- Dependent on external libraries (gsread, RGB matrix, Pillow)
- Must run with sudo for matrix control
- Limited by Python performance for real-time updates

Network Constraints

- Requires stable WiFi for Google Sheets access
- Delays if internet connection is slow or unavailable

Display Constraints

- Resolution limited by total panel configuration
- Text formatting must fit within panel layout

Overall Constraints

- Budget
- Mounting space
- Semester deadline

Problem Overview

The Fletcher Jones Foundation Makerspace at the University of the Pacific is a hands-on S.T.E.A.M. facility for interdisciplinary design and innovation, managed under the School of Engineering and Computer Science. The space is open to all students, faculty, and staff, and it is never reserved, even during events or classes. This open-access model creates a high-demand service environment where real-time operational communication is critical.

Through her direct experience working at the Makerspace, team member Anna Noto identified several persistent operational challenges. Communication was fragmented across multiple informal channels, leading to inconsistency in the information available to visitors. Operational updates such as machine shop closures, 3D printer queue status, and available tool counts changed frequently and could not be effectively communicated using static signage. Visitors routinely asked staff questions about resource availability that could easily be answered by a persistent display, adding unnecessary pressure to staff during high-traffic periods. The lack of any real-time visibility into space status resulted in system inefficiencies and bottlenecks in staff support.

The M.I.N.D. project was proposed to directly address these challenges by replacing fragmented, informal communication with a centralized, always-on, real-time LED display driven by staff-managed content in Google Sheets. This approach builds on the Makerspace's existing workflows without requiring additional training or infrastructure changes.

Background

RGB LED matrix panels are modular display units composed of individually addressable RGB LEDs arranged in a grid. Each panel in this project measures 64x64 pixels. When multiple panels are connected in a daisy-chain configuration, they behave as a single extended canvas. Control signals are transmitted serially through a HUB75 interface, a standard 16-pin ribbon connector that carries row and column address lines, clock, latch, and output-enable signals. The `rpi-rgb-led-matrix` library, developed by Henner Zeller, provides an optimized low-level interface between Raspberry Pi GPIO pins and HUB75-compatible panels.

The Raspberry Pi 4B serves as the central processing unit for this project. It runs Raspberry Pi OS 64-bit and interfaces with the LED panels via its GPIO header through an Adafruit RGB LED Matrix Bonnet, which handles signal buffering and level shifting between the Pi's 3.3V GPIO outputs and the panels' 5V logic inputs. The Pi also connects to Google Sheets via the `gspread` Python library, which authenticates using a Google Cloud service account and reads cell data from a shared spreadsheet over WiFi.

The Python Imaging Library (Pillow) is used to render text and other content as rasterized pixel images. Because the LED matrix library accepts images as input, the pipeline converts Google Sheets cell values into Pillow images, which are then mapped to the physical panel layout and pushed to the display. This separation of content management (Google Sheets), image rendering (Pillow), and hardware control (`rpi-rgb-led-matrix`) creates a modular and maintainable architecture.

Power for the LED panels is supplied by a Mean Well LRS-600-5 switching power supply, which converts 120V AC from a standard wall outlet to regulated 5V DC at up to 120A. Each LED panel draws approximately 4A at full brightness (approximately 20W), and the 18-panel system has an estimated full-load draw of approximately 360W, well within the 600W supply capacity. A 30A main fuse protects the entire 5V system, and three 15A branch fuses provide per-row overcurrent protection.

Design Implementation

Team Roles

Anna Noto: Hardware-Software Integration & Panel Construction, Project Manager

Matthew Bernardino: Software & Wiring

Jake Tandy: Power System

System Structure

The M.I.N.D. system is organized into three major subsystems: the hardware subsystem (power, LED panels, mounting, and enclosure), the software subsystem (Raspberry Pi interface and display pipeline), and the optional user interaction subsystem (touchscreen controller). These subsystems were developed concurrently using a phased integration approach that validated core functionality incrementally before assembling the full system.

Hardware Description

LED Panels and HUB75 Signal Path

The LED panels are connected using the HUB75 signal path, where the Raspberry Pi sends display data through the Adafruit HAT and into the first panel. From there, the panels use a daisy-chain connection, meaning the data goes into the DATA IN port of the first panel and then leaves through the DATA OUT port to continue into the next panel. This allows our panels to act as one larger display instead of individual displays. Each individual panel is 64×64 pixels, and when multiple panels are connected together, their pixel areas combine to create the full display size. For the final design, the panels are arranged into 3 rows of 6 chained panels so the software can treat them as one large canvas while still sending the correct signal path through each physical panel.

Power System

The power system for this senior project consists of fuses, wires, terminal blocks, and the power supply. The LRS-600 power supply is the core of the system, providing a steady 5V DC to the panels through a connection of fuses and terminal blocks. The panels are rated at 5V and 4A, putting their power draw at 20W per panel. Considering all panels, the total draw from the panels is 360W. We went with the LRS-600 to have power headroom, as it is rated for 600W. This also allows for any expansions or additions to be made to the system in the future. The fuses are rated at 30A, providing overcurrent and short protection. Each line of panels being rated for 24A, we decided to go with the 30A fuse to secure each line of panels. The grounding for the power system is handled through the same terminal blocks that the distribution lines use, obviously just using different terminals. Overall, the system is set up well for reliability and safety for the foreseeable future.

Mounting and Enclosure

The LED panels are mounted to a frame constructed from aluminum extrusions cut to the correct length using the Makerspace horizontal bandsaw. Each panel is secured to the extrusions using M3 flanged screws threaded through a spacer and washer combination, which utilize the panels' built-in mounting holes and allow them to slide onto the aluminum extrusion profiles for a secure, adjustable fit. To prevent gaps between panels within each row and ensure a flush, unified display surface, custom mounting brackets were designed in Autodesk Fusion and 3D printed in-house. These brackets clamp adjacent panels together and maintain consistent panel-to-panel spacing across the full display. The aluminum extrusions are joined to each other using aluminum corner brackets, which provide structural rigidity to the overall frame.

The Raspberry Pi 4B (housed in its aluminum alloy enclosure with the active cooling fan attached) and the Mean Well power supply are both mounted directly to the rear of the frame using the same spacer-washer-screw method, keeping all electronics consolidated within the display structure. The rear of the frame is intentionally left open to ensure adequate airflow for thermal management and to allow easy access to wiring, connections, and components for maintenance or future modifications.

The entire assembly was designed with permanent wall mounting as the intended end state. The aluminum extrusion frame provides the necessary structural rigidity for wall installation, and the consolidated rear-mounted electronics ensure that a single wall-hung unit contains everything needed for full operation with only a power cable and network connection required.

Software Description

Google Sheets Integration

The Google Sheets integration is the input side of the software. The sheet stores the messages that need to update on the display, and the program uses gspread with a google service account key to access those cells. This allows the display content to be updated without changing the code directly. When a user edits the Google Sheet, the Raspberry Pi can read the updated values and run them through the pipeline for the display to then update with.

Display Pipeline

The display pipeline is the process that turns the Google Sheets data into something the LED matrix can show. After the text is read from the sheet, the software formats each message for the correct row or section of the display. Pillow is then used to draw the text onto an image based on the full panel size. That image is passed to the RGB matrix library, which sends the pixel data to the LED panels to then show the message.

Controller Module

The system includes a Kivy-based touchscreen controller running on a dedicated Raspberry Pi 3B that allows staff to update display content through a 5" Elecrow touchscreen interface. User inputs through the GUI update directly to Google Sheets, which then triggers the existing display pipeline on the main system. The controller functions independently and wirelessly, powered by a 12V 10000mAh lithium battery with a DROK buck converter stepping the voltage down to 5V for the Pi to work properly. It uses WiFi connectivity to function and automatically checks the Google Sheet every 15 seconds to stay synchronized with updates. The application runs as a fullscreen kiosk on boot, with an estimated battery life of 6–7 hours under normal use.

Timeline

Weeks 1–2

- Onboard new team members and establish shared understanding of project goals and constraints
- Assign primary leadership roles while maintaining collaborative responsibility across subsystems
- Retrieve and inspect LED panels purchased in the previous semester
- Review and revise prior semester design decisions

Weeks 3–4

- Develop and refine initial materials list
- Evaluate alternative hardware and software design approaches
- Set up Raspberry Pi 3B+ environment (OS, libraries, dependencies)
- Perform GPIO pin mapping between Raspberry Pi and one LED panel
- Finalize materials list and obtain approval to order components
- Coordinate with faculty to request and justify additional funding
- Analyze use of AI-assisted design reviews for identifying weaknesses and risks

Weeks 5–6

- Receive and inventory ordered components
- Transition system to Raspberry Pi 4B after successful display validation
- Establish power system and adjust as we increase the number of panels
- Expand GPIO and wiring design to support multi-panel configurations
- Implement proof-of-concept display output across multiple panels
- Begin enclosure and mounting concept validation

Weeks 7–8

- Refine display driver code for stability and error handling
- Develop content formatting standards (text size, layout, timing)
- Synchronize output across panels

Weeks 9–10

- Integrate user interaction and configuration options
- Conduct system-level testing and debugging
- Panel measurements and cuts made for mounting of display

Weeks 11–12

- Perform full system validation and AI-assisted design review
- Address identified risks and weaknesses
- Full display assembly
- Power system establishment and full component testing
- Incorporation of 3D printed custom brackets
- Testing of solved issue with pillow data
- Final AI assessment to address any limiting issues for final testing

Weeks 13–14

- Final walk throughs and tests to ensure safe and proper operation
- Final documentation and preparation for deployment
- Final report and demonstration

Verification Procedure

The system was verified through a series of incremental tests performed at each stage of integration, progressing from single-component validation to full-system testing. This phased approach allowed defects to be identified and resolved in isolation before they could compound in a more complex system state.

GPIO pin mapping was verified by connecting the Raspberry Pi 3B+ to a single LED panel through the Bonnet and running static test pattern scripts. Correct illumination of known pixel patterns confirmed accurate wiring and pin assignments. This test was then repeated after migrating to the Raspberry Pi 4B to confirm consistent behavior on the target hardware. Following single-panel validation, a second panel was daisy-chained and the test patterns were extended across both panels to verify correct HUB75 chain behavior.

Power supply testing was performed by measuring the supply output voltage with a multimeter before and after connecting panels. The supply was verified to output a stable 5V under zero load, single-panel load, and two-panel load, confirming stable AC-to-DC conversion. Fuse continuity was verified across all three branch fuses using a continuity tester before the system was energized. Grounding was checked at each panel ground connection to ensure a common return path.

WiFi connectivity was tested by attempting to connect the Raspberry Pi to the university's PacificDeviceReg network. Initial connection attempts failed, and the issue was traced to the university's MAC address registration requirement. The Raspberry Pi's MAC address was provided to Pacific Technology, which whitelisted it on the network. Subsequent connection attempts succeeded and the Pi was able to authenticate with Google Sheets.

The mounting hardware was validated by performing fitment tests with M3 flanged flat-top screws, washers, and spacers on the aluminum extrusion profiles before committing to the final mounting configuration. The custom 3D-printed brackets were test-fitted on an assembled two-row section before final installation. Bracket dimensions were verified by measuring the panel-to-panel spacing and comparing against Fusion design files.

Software pipeline testing was performed by feeding known test data into the Google Sheet and observing the display output. The Pillow text-to-image conversion was first validated in simulation by rendering text to a PNG file and verifying pixel coordinates against expected values. The full pipeline was then validated on the physical display by updating specific cells in the Google Sheet and confirming that the correct text appeared in the correct region of the display within the poll interval.

System-level testing was performed with all 18 panels assembled, fully wired, and running the production code. The display was observed for stability over multiple operating sessions, confirming continuous operation without crashes.

Evaluation

The following table summarizes the test results obtained during system verification, with comparison against the original project specifications.

Test	Result	Specification	Met?
GPIO pin mapping - single panel	Successful communication and correct display output on single panel	Reliable GPIO signal integrity	Yes
Daisy-chain - two panels	Both panels operated as a single canvas with correct content scaling	Multi-panel sync	Yes

Power supply output	Verified 5V stable; 0.207A with 1 and 2 panels; AC-to-DC stable	5V regulated	Yes
Fuse continuity	All three branch fuses passed continuity check	Overcurrent protection per row	Yes
WiFi/Google Sheets connectivity	Pi MAC address whitelisted; gspread authenticated successfully	Network connectivity for data fetch	Yes
Pillow text rendering	Text correctly rendered to 64x64 pixel canvas and confirmed via PNG preview	Correct text-to-pixel conversion	Yes
Multi-panel content scaling	Global variables corrected text scaling across full 18-panel 384x192 canvas	Display scales with chain/parallel config	Yes
Full system assembly	All 18 panels mounted, wired, powered, and displaying live content	Complete functional display	Yes
Controller write-back to Google Sheets	Button presses on Kivy UI updated correct cells; display refreshed on next poll	Controller triggers display update	Yes

Table 1: Summary of test results compared against project specifications.

All major system specifications were met. The power supply delivered stable 5V output well within its rated capacity, and the 360W estimated load leaves approximately 240W of headroom. The display pipeline successfully reads from Google Sheets, renders text to the correct pixel dimensions, and updates the full 18-panel display. The optional controller correctly writes back to the spreadsheet and triggers display updates. Continuous operation testing confirmed stable system behavior across multiple sessions without crashes or display artifacts.

Analysis

The M.I.N.D. system successfully achieved its primary design objective: a permanently installed, always-on LED display that provides real-time Makerspace operational information from a Google Sheets data source. The modular LED panel architecture proved to be a sound design choice, allowing the team to validate the system incrementally from a single panel to the full 18-panel configuration without redesigning the hardware or software interfaces.

The decision to use a Google Sheets backend was deliberate and well-justified. It eliminated the need for staff to learn a new system, leveraged tools already in daily use at the Makerspace, and provided a low-friction path for content updates. The optional Kivy touchscreen controller further reduced the friction of routine updates by eliminating the need to open a browser or navigate spreadsheet cells, instead presenting a purpose-built UI with clearly labeled controls for each data point.

The power system design performed within specifications. The Mean Well LRS-600-5 supply provided stable 5V output throughout testing, and the per-row fusing architecture provides meaningful overcurrent protection for each panel section. The thermal management solution, combining an aluminum alloy enclosure for a passive cooling backing and active fan cooling via the USB power tap maintained safe operating temperatures for the Raspberry Pi.

The major challenge encountered during the project was the transition from one semester team to another, which effectively required a significant design restart rather than a continuation of prior work. Inherited documentation was found to contain inaccuracies in wiring and power planning, necessitating a full re-evaluation of the hardware architecture in the early weeks of the semester. This experience reinforced the importance of rigorous documentation and handoff procedures in engineering projects intended for long-term deployment.

A secondary challenge was the WiFi connectivity requirement. The university network required MAC address registration, which was not anticipated during initial planning and caused a delay in software testing. This issue was resolved through coordination with Pacific Technology staff. Future iterations of this system may benefit from using a dedicated network connection or a local network fallback.

In summary, the project met or exceeded the majority of its specifications and has produced a system that is ready for permanent installation and long-term use in the Makerspace.

Budget

The following table lists all items purchased, borrowed, or otherwise used in the project. Items borrowed from the Makerspace or available from prior semester inventory are noted accordingly.

Item	Qty	Unit Price	Total	Source
LED Panels 64x64	18	\$29.75	\$535.50	Amazon
Mean Well USA Inc. LRS-600-5 Power Supply	1	\$88.50	\$88.50	Supplier
RGB Triple LED Matrix Bonnet (Adafruit)	1	\$9.95	\$9.95	Adafruit

Aluminum Extrusion Profile 47.2 in, T-Slot 2040	2	\$49.00	\$98.00	Amazon
Aluminum Extrusion Profile 47.2 in, T-Slot 2020	1	\$40.04	\$40.04	Amazon
Usnell 4060 Aluminum Corner Bracket w/ screws	1	\$19.56	\$19.56	Amazon
FUZE AUTO STF40 12AWG / BLADE X10 Fuse Holder	3	\$7.55	\$22.65	Amazon
HOOK-UP STRING 12AWG Red / Black Meter	4 ea	\$4.74	\$18.96	Supplier
Red Meter (wire)	2	\$5.54	\$11.08	Supplier
Lever Mule Connector / Assorted Spring Connectors	1 pk	\$5.97	\$5.97	Amazon
M3 x 12mm Socket Flanged Button Screws	100	\$0.21	\$21.00	Amazon
Raspberry Pi 4B (4GB)	1	~\$55.00	~\$55.00	Donation
Raspberry Pi 3B (Controller)	1	Borrowed	\$0	Makerspace
Elecrow 5" LCD Touchscreen (Model B)	1	Borrowed	\$0	Donation
12V 10000mAh Battery (Controller)	1	Borrowed	\$0	Makerspace
DROK Buck Converter	1	Borrowed	\$0	Makerspace
Vilros Fan (Raspberry Pi Cooling)	1	Borrowed	\$0	Donation
Miscellaneous (spacers, washers, heat shrink)	Var.	~\$10.00	~\$10.00	Makerspace /Amazon
TOTAL (approximate)			~\$936	

Table 2: Project materials, quantities, unit prices, and sources.

Labor Estimates

Hardware Setup

- Panel mounting, PSU wiring, HUB75 connections
- 15 - 20 hours

Software Development

- Google Sheets integration, display pipeline (gsread, Pillow, RGB Matrix Libraries), formatting display
- 20-25 hours

Debugging & Testing

- Panel mapping, signal issues, refresh timing
- 10–15 hours

System Integration

- Hardware + Software working together and verification
- 8–12 hours

Documentation

- Report, diagrams, presentation
- 5–8 hours

Global, Social, & Environmental Impact**Social Impact**

The M.I.N.D. system directly improves the working environment for Makerspace staff by reducing the number of repetitive informational questions directed to staff during high-traffic periods. By presenting operational information such as machine shop status, available tools, operating hours, and 3D printer queue prominently and in real time, the display empowers students and visitors to self-serve routine inquiries. This is particularly meaningful in a high-demand academic STEAM environment where staff time is limited and the space serves an interdisciplinary community spanning multiple schools and disciplines.

Beyond the Makerspace, the design pattern established by this project is broadly applicable to similar shared-resource environments: university libraries, computer labs, fabrication studios, and collaborative workspaces. The use of a Google Sheets backend specifically makes this architecture accessible to non-technical operators, lowering the barrier to adoption for institutions that lack dedicated IT resources. In this sense, the project contributes to a broader movement toward democratized information display technology in educational institutions.

Economic Impact

The project was completed at a total materials cost of approximately \$1,024, a fraction of the cost of a commercial digital signage solution of comparable size. Commercial RGB LED video walls of similar specifications typically cost several thousand dollars when purchased as integrated turnkey systems, and even more when professional installation is included. By using commodity hardware components, open-source software libraries, and student labor, the M.I.N.D. system demonstrates that high-quality digital signage infrastructure can be built and maintained at significantly lower cost within an institutional setting.

The system also has indirect economic value for the Makerspace by reducing wasted staff time spent answering repetitive informational questions. Over the operating lifetime of the display, even modest improvements in staff efficiency translate to meaningful cumulative time savings. The modular design also means that future expansions or repairs can be performed incrementally without replacing the entire system.

Environmental Impact

The LED panel technology used in this project is significantly more energy-efficient than comparable legacy display technologies such as incandescent or fluorescent signage. LEDs produce light through electroluminescence rather than thermal emission, resulting in far lower energy consumption per unit of luminous output. The system's estimated 360W operating draw is well within the energy budget of a standard wall outlet, and the use of a highly efficient switching power supply (the Mean Well LRS-600-5 has a typical efficiency greater than 88%) minimizes conversion losses.

The modular design of the system also reduces long-term electronic waste. If an individual LED panel fails, only that panel needs to be replaced rather than the entire display. The Raspberry Pi compute unit is likewise replaceable independently of the display hardware. This repairability-first design philosophy aligns with principles of sustainable electronics engineering. The aluminum mounting extrusions and custom brackets are durable materials that will not need to be replaced under normal operating conditions.

From a broader sustainability perspective, the project contributes to the efficient operation of a shared resource space. By enabling better real-time communication of Makerspace policies and tool availability, the display helps ensure that the space's equipment is used appropriately and that resources are not wasted due to visitor confusion about operating procedures.

Future Work

Performance & Speed

System performance could be improved by upgrading to a Raspberry Pi 5, which offers meaningfully faster CPU and memory performance and has confirmed compatibility with the Adafruit HAT. Switching to a headless OS configuration would further reduce background resource consumption.

Display Scale & Content

The modular panel architecture makes display expansion straightforward. Adding additional panels to the daisy-chain simply requires updating the `chain_length` and parallel global variables and adjusting the Google Sheets cell map. Future work could explore adding a fourth row of panels to increase the total display height, or using smaller pixel-pitch panels (such as P1.875 or P1.5) for improved readability at greater viewing distances.

On the content side, future improvements could include image and graphic support, allowing the display to show logos, icons, or diagrams alongside text. Support for multiple display modes

(e.g., scrolling announcements, static status, event schedule) with timed switching would also increase the utility of the display for different operating scenarios.

Additional Software Integrations

Several software integration improvements are identified as high-value future work. Integration with 3D printer APIs would enable automatic updates of printer status and queue information without manual staff input. A real-time notification system using Discord or Slack bots could alert staff to system errors or connectivity issues. AI-assisted message formatting could automatically clean and shorten entered text to fit the display dimensions optimally.

Conclusions

The Makerspace Information & Notification Display (M.I.N.D.) project successfully delivered a permanently installed, real-time LED information display for the Fletcher Jones Foundation Makerspace at the University of the Pacific. The system meets its core design objectives: it displays real-time operational information sourced from a Google Sheets backend, supports a full 18-panel 384x192 pixel display area, operates continuously on a 5V power system with appropriate overcurrent protection, and provides an optional touchscreen controller for staff use.

The project demonstrated that complex embedded systems can be built effectively from commodity hardware and open-source software when the architecture is carefully designed for modularity and incremental validation. The challenges encountered such as the semester handoff, network connectivity hurdles, and mounting hardware iterations were all resolved through systematic troubleshooting and close collaboration with Makerspace staff and faculty advisors.

The result is a system that will continue serving the Makerspace community beyond the scope of this course, reducing staff workload, improving visitor awareness, and providing a foundation for future enhancements. The team is proud to have delivered not a prototype, but a lasting installation to leave behind as they graduate.

Acknowledgements

The team would like to express sincere gratitude to the following individuals whose guidance, resources, and support made this project possible.

Dr. Don Lee: Faculty advisor and course instructor, for project guidance, funding support, and technical feedback throughout the semester.

Mark Forman: Makerspace Electronics Technician, for facilitating component procurement, providing access to Makerspace tools and equipment, and double checking our wiring.

Jeremy Hanlon: Makerspace Manager, for welcoming the permanent installation of the board in the Makerspace, hands-on assistance with mounting design, bandsaw operation, and hardware consultation.

Roy Torres: Pacific Technology, for assisting with Raspberry Pi MAC address registration on the university network.

Dr. Vivek Pallipuram: For giving the original approval of the project.

SOECS Department: For providing additional funding as well as the resources and environment to support senior design projects.

References

DeVito, James. “DIY LED Video Wall.” *Adafruit Learning System*, 18 July 2013,

learn.adafruit.com/adafruit-diy-led-video-wall/frame.

“Gspread — Gspread 4.0.0 Documentation.” *Docs.gspread.org*,

docs.gspread.org/en/latest/.

Pillow. “Pillow — Pillow (PIL Fork) 6.2.1 Documentation.” *Readthedocs.io*, 2011,

pillow.readthedocs.io/en/stable/.

“Raspberry Pi Documentation - Raspberry Pi OS.” *Www.raspberrypi.com*,

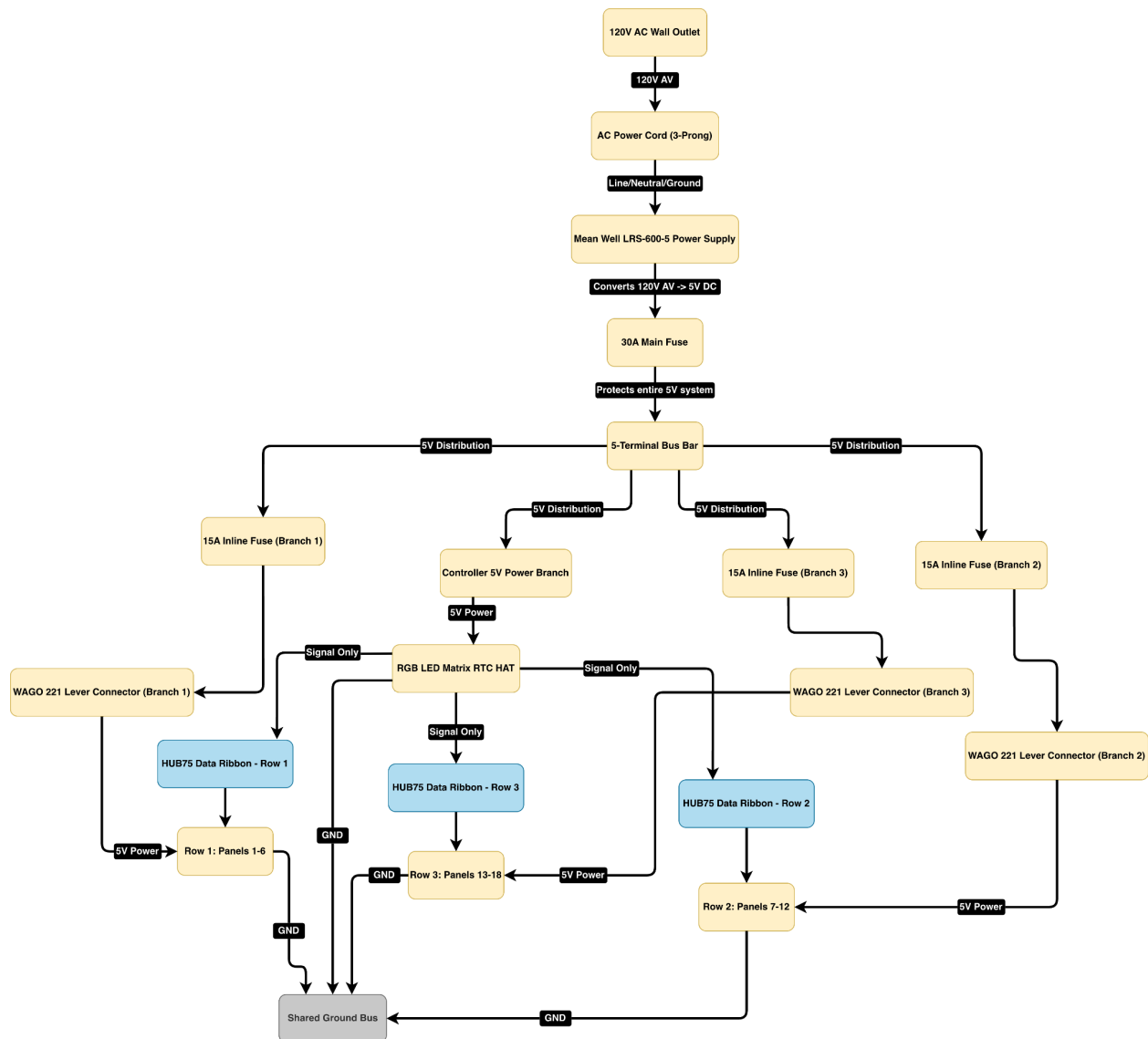
www.raspberrypi.com/documentation/computers/os.html#introduction.

Zeller, Henner. “Controlling RGB LED Display with Raspberry Pi GPIO.” *GitHub*, 2 May 2022,

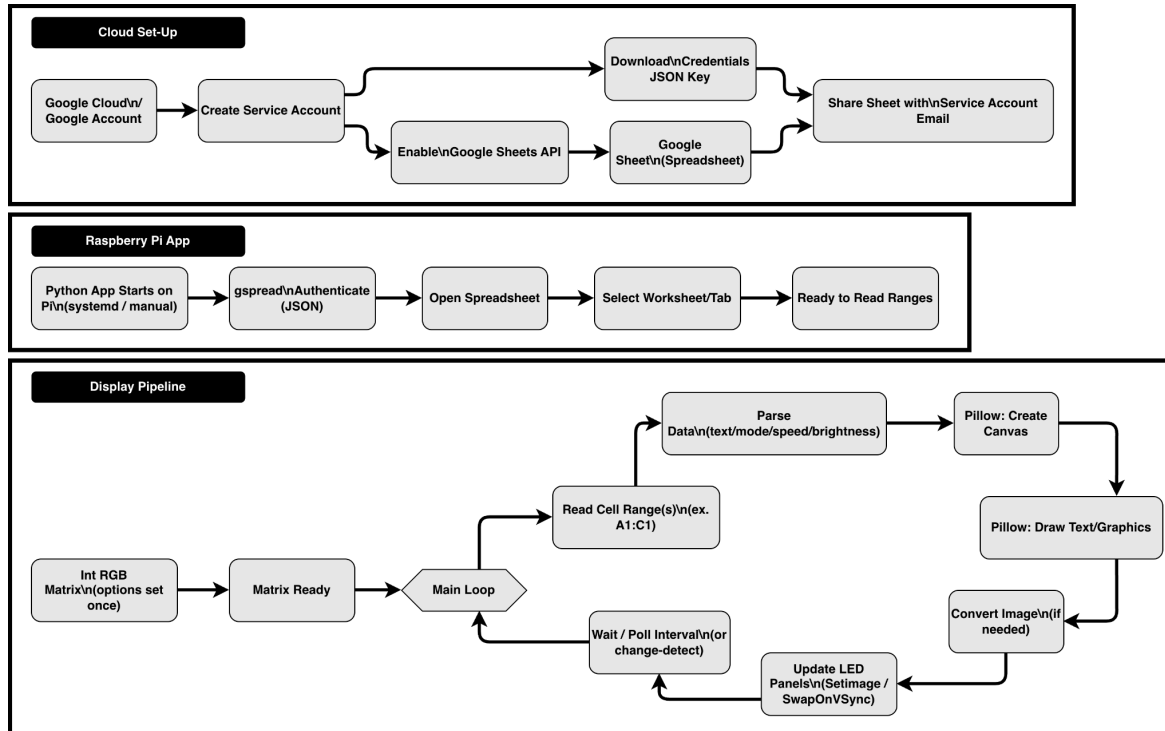
github.com/hzeller/rpi-rgb-led-matrix.

Poster

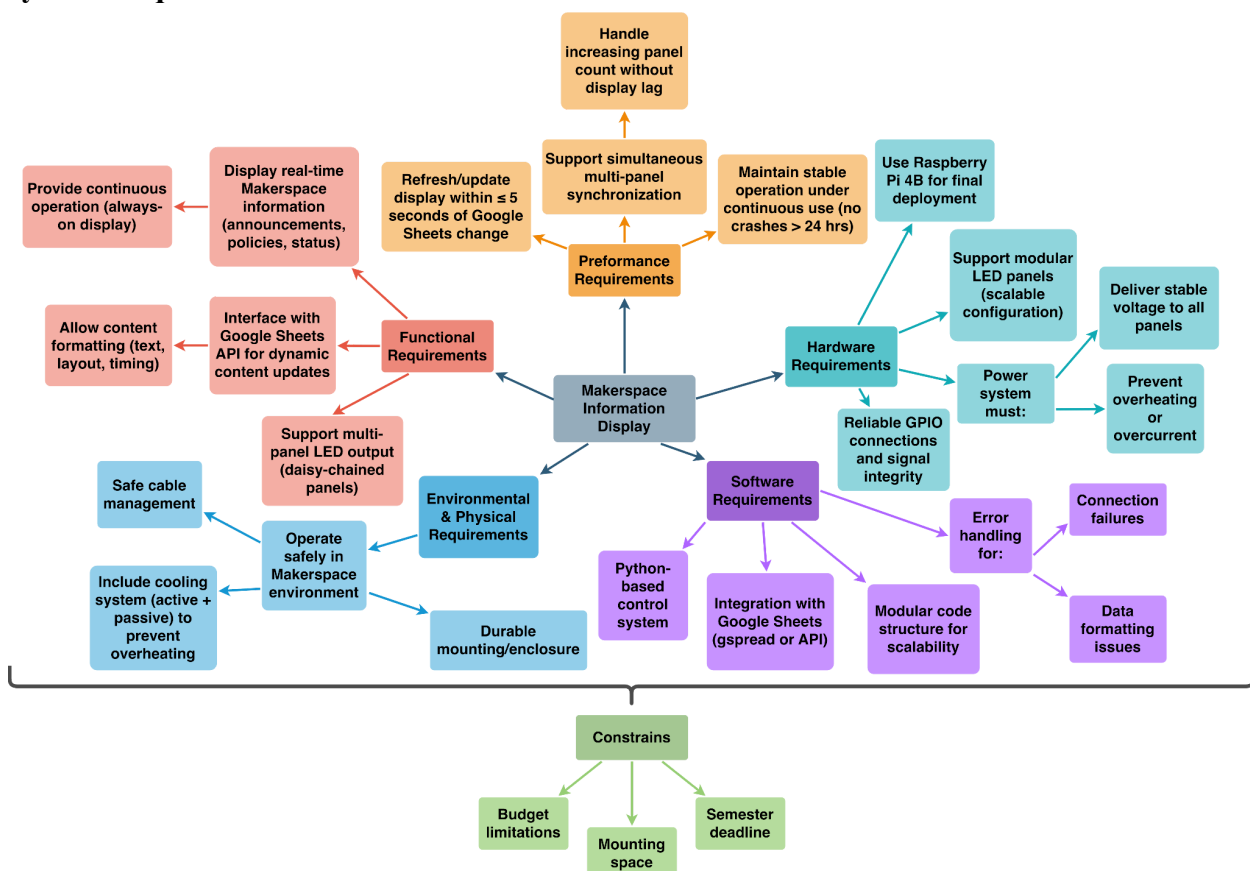
Power System Flow Chart:



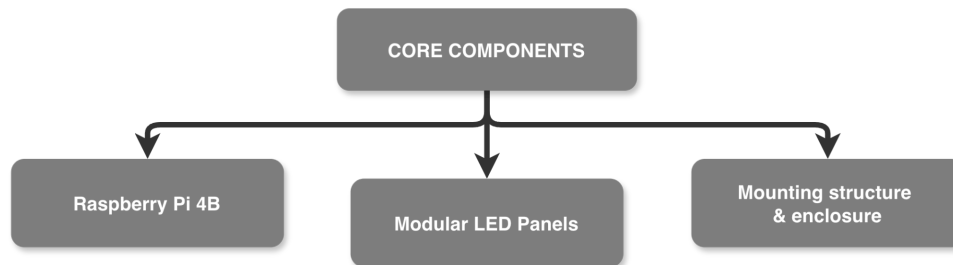
Program System Flow Charts:



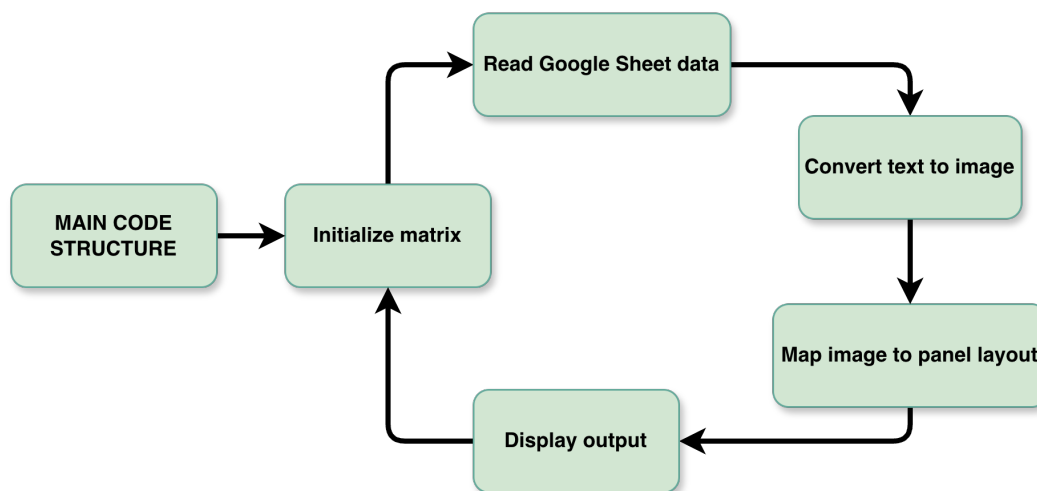
System Requirements:



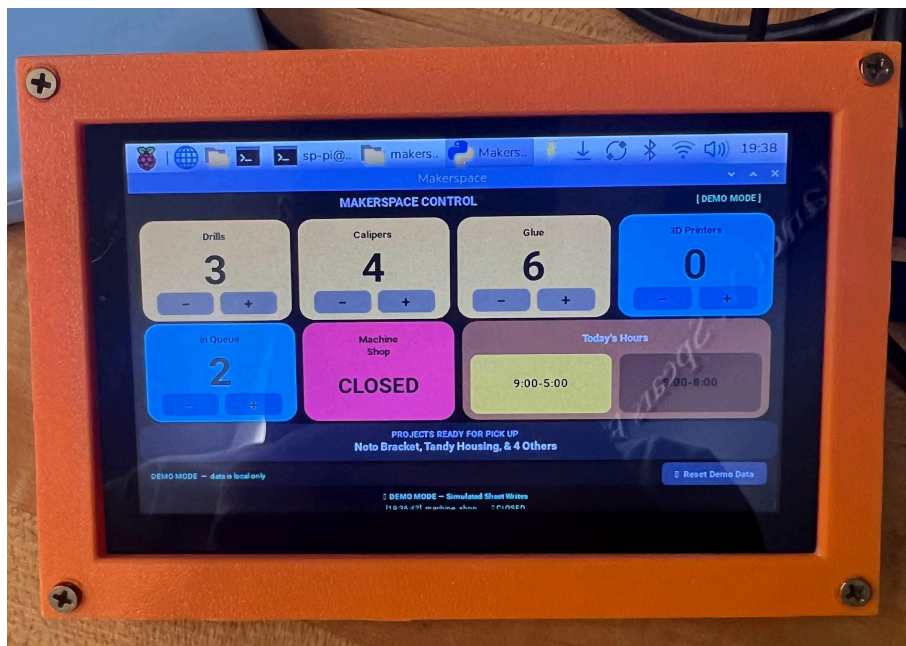
Hardware Design Details:



Main Code Structure:



Controller Demo:



Code Examples:

```
import gspread

gc = gspread.service_account(filename='LEDservice_account.json')

sh = gc.open("LED Panel Controller")

print(sh.sheet1.get('A1:C1'))
print('\n')
```

Output: [['Machine Shop: ', 'OPEN', '3D Printer 1 Status: 72%']]

gspread_example.py

```
from PIL import Image, ImageDraw, ImageFont
import numpy as np

def text_to_pixel_data(message: str, width: int = 64, height: int = 64):
    # Create blank image
    img = Image.new("RGB", (width, height), (0, 0, 0))
    draw = ImageDraw.Draw(img)

    # Load font
    font = ImageFont.load_default()

    # Draw text
    draw.text((2, 20), message, font=font, fill=(255, 255, 255))

    # Convert to numpy array for matrix-style processing
    pixel_array = np.array(img)

    return img, pixel_array

# Example use
img, pixel_data = text_to_pixel_data("Hello")

# Save image so you can verify what Pillow created
img.save("text_preview.png")

# Show shape of the pixel array
print("Pixel array shape:", pixel_data.shape) # (64, 64, 3)

# Show a small sample of pixel data
```

```
#print("Rows 18-29 and Cols 0-29 sample:")
#print(pixel_data[18:30, 0:30])
```

```
# Simple on/off preview in terminal
print("\nText preview:")
for y in range(64):
    row = ""
    for x in range(64):
        if np.any(pixel_data[y, x] > 0):
            row += "#"
        else:
            row += "."
    print(row)
```

Output:

```
.....
.#...#.....#.#.....
.#...#.....#.#.....
.#...#####.#.#.#####
.#...#.#.#.#.#.#.#.#.#
#####.#####.#.#.#.#
.#...#.#.#.#.#.#.#.#.#
.#...#.#.#.#.#.#.#.#.#
.#...#.#.#.#.#.#.#.#.#
.#...#.#.#.#.#.#.#####
.....
```

pillow_example.py (Output shows how the led panel would display it)

```
import gspread
from PIL import Image, ImageDraw, ImageFont
import time
from rgbmatrix import RGBMatrix, RGBMatrixOptions

# Sets variables for matrix options
# Used to make pixel window size adjust with these variables
rows = 64
cols = 64
chain_length = 1
parallel = 1

def setup_matrix():
    options = RGBMatrixOptions()
```



```
options.rows = rows
options.cols = cols
options.chain_length = chain_length
options.parallel = parallel
options.hardware_mapping = 'regular'
options.gpio_slowdown = 4
return RGBMatrix(options=options)

"""
    Connect to Google Sheets using a service account JSON file
    and return the values from the requested range as a list.
"""
def read_google_sheet(json_keyfile: str, sheet_name: str, cell_range: str):

    gc = gspread.service_account(filename=json_keyfile) # Authentication
    sh = gc.open(sheet_name) # Open the Google Sheet
    ws = sh.sheet1 # Access the first sheet

    # Read the specified cell range via c_range
    values = ws.get(cell_range)

    # Flatten to make easier to convert with pillow
    clean_msg = [row[0].strip() if row else "" for row in values]

    return clean_msg

"""
    Using google sheets B1:B3 cell values, place the text below one another.
"""
def text_to_image(cell_values, width: int, height: int):

    img = Image.new("RGB", (width, height), (0, 0, 0)) # Create blank black image
    draw = ImageDraw.Draw(img) # Create drawing context

    font = ImageFont.load_default() # Load default font

    # Starting position for text
    y_position = 5

    # Loop through the cell values and draw each on a new line
    for text in cell_values:
```

```
        draw.text((2, y_position), text, font=font, fill=(255, 255, 255))
        y_position += 15 # Move the position down for the next line

    return img

# Main function to test the pipeline
def main():

    # Define the Google Sheets JSON key file and sheet name
    json_keyfile = "LEDservice_account.json"
    sheet_name = "LED Panel Controller"
    c_range = "B1:B3"

    #Total display height and Total display width to be used in the text_to_image
function
    p_height = rows * parallel
    p_width = cols * chain_length

    # Step 1: Read B1:B3 from Google Sheets
    msg = read_google_sheet(json_keyfile, sheet_name, c_range)
    print("Values read from sheet:", msg) # print the message

    # Step 2: Display the separate values on the image (each in its own line)
    img = text_to_image(msg, width=p_width, height=p_height) #insert msg into pillow
function
    print("\nImage processed, going to rgb matrix init handoff")

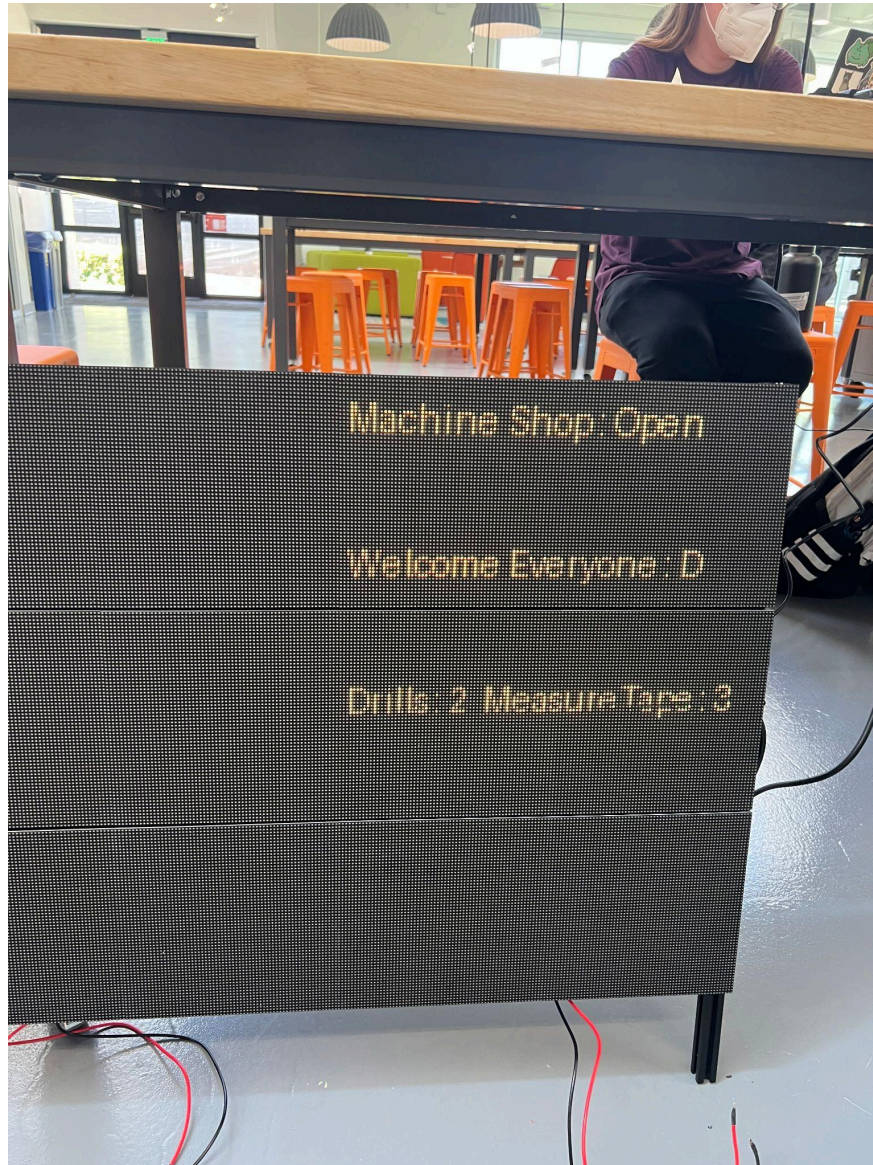
    # Step 3: Display the image and verify
    matrix = setup_matrix()
    matrix.SetImage(img.convert("RGB"))

    try:
        print("Displaying on matrix. Press CTRL-C to exit.")
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        print("Exiting.")
        matrix.Clear()
```

```
if __name__ == "__main__":  
    main()
```

Current Iteration of led_matrix.py (April 14, 2026)

Reads google sheet cell data, converts to pixels, hands off to rgb matrix library, displays on board. Scalable with chain_length and parallel values



Output of current code iteration (April 14, 2026)

```
CELL_MAP = {
    "drills":      "B3",
    "calipers":    "B4",
    "glue":        "B5",
    "printer_open": "E3",
    "printer_queue": "E4",
    "machine_shop": "B7",
    "machine_reopen": "B8",
    "hours_session1": "D8",
    "hours_session2": "E8",
    "projects":    "B11",
}
```

makerspace_controller.py: CELL_MAP dictionary maps each controller element to a specific cell in the Google Sheet

```
self.drills_tile = CounterTile("Available\nDrills", "drills",
...)
self.calipers_tile = CounterTile("Available\nCalipers", "calipers",
...)
self.glue_tile = CounterTile("Available\nGlue", "glue",
...)
self.printer_tile = CounterTile("Available\n3D Printers", "printer_open",
...)
self.queue_tile = CounterTile("3D Prints\nin Queue", "printer_queue",
...)
```

makerspace_controller.py: How button labels are created

```
POLL_INTERVAL = 15 #seconds between auto-refresh from sheet
```

makerspace_controller.py: How the auto-refresh is set for the controller.

Item	Qty	Unit Price	Total	Source
LED Panels 64x64	18	\$29.75	\$535.50	Amazon
Mean Well USA Inc. LRS-600-5 Power Supply	1	\$88.50	\$88.50	Supplier
RGB Triple LED Matrix Bonnet (Adafruit)	1	\$9.95	\$9.95	Adafruit
Aluminum Extrusion Profile 47.2 in, T-Slot 2040	2	\$49.00	\$98.00	Amazon
Aluminum Extrusion Profile 47.2 in, T-Slot 2020	1	\$40.04	\$40.04	Amazon
Usnell 4060 Aluminum Corner Bracket w/ screws	1	\$19.56	\$19.56	Amazon
FUZE AUTO STF40 12AWG / BLADE X10 Fuse Holder	3	\$7.55	\$22.65	Amazon
HOOK-UP STRING 12AWG Red / Black Meter	4 ea	\$4.74	\$18.96	Supplier
Red Meter (wire)	2	\$5.54	\$11.08	Supplier
Lever Mule Connector / Assorted Spring Connectors	1 pk	\$5.97	\$5.97	Amazon
M3 x 12mm Socket Flanged Button Screws	100	\$0.21	\$21.00	Amazon
Raspberry Pi 4B (4GB)	1	~\$55.00	~\$55.00	Donation
Raspberry Pi 3B (Controller)	1	Borrowed	\$0	Makerspace
Elecrow 5" LCD Touchscreen (Model B)	1	Borrowed	\$0	Donation
12V 10000mAh Battery (Controller)	1	Borrowed	\$0	Makerspace
DROK Buck Converter	1	Borrowed	\$0	Makerspace
Vilros Fan (Raspberry Pi Cooling)	1	Borrowed	\$0	Donation
Miscellaneous (spacers, washers, heat shrink)	Var.	~\$10.00	~\$10.00	Makerspace /Amazon
TOTAL (approximate)			~\$936	